

This is unreleased documentation for Serverpod **Next** version.

For up-to-date documentation, see the [latest version](#) (4.0.0).

3..43

EXCEPTION MONITORING

 Copy as Markdown

Exception monitoring

When something throws in production you want to hear about it without reading log tables. Serverpod can hand every exception to code you provide, so you can forward it to a monitoring service such as [Sentry](#), [Highlight](#), or [Datadog](#).

This works for exceptions thrown in your own code and for exceptions the framework raises itself, including failures during startup and shutdown.

WARNING

This is an experimental feature, reached through `serverpod.experimental`. Its API can change in a breaking way in any minor release, so pin your Serverpod version if you depend on it and re-check this page when you upgrade. Experimental APIs move to `Serverpod` proper once they settle.

Add a handler

Handlers are registered through the `experimentalFeatures` argument on the `Serverpod` constructor. A handler receives the event, the space it came from, and its context:

```
var pod = Serverpod(  
  args,  
  experimentalFeatures: ExperimentalFeatures(  
    diagnosticEventHandlers: [  
      AsEventHandler((event, {required space, required context}) {  
        print('$event  Origin is $space\n Context is ${context.toJson()}');  
      }  
    ],  
  ),  
);
```



Ask AI

The `AsEventHandler` class wraps a plain function as a handler, which is the shortest way to get started. For anything longer, implement `DiagnosticEventHandler` yourself. The `space` tells you whether the event came from your application code or from the framework, and the `context` carries details such as the server ID and run mode.

Add as many handlers as you like. Serverpod runs them concurrently and does not wait for them, so a slow handler does not slow down the request that triggered it. Each invocation is abandoned after `experimentalDiagnosticHandlerTimeout`, which defaults to 30 seconds. See the [Configuration reference](#).

Handlers are for observation only. They cannot suppress an exception, change it, or alter the response. If a handler throws, the error is logged and otherwise ignored.

Report an exception yourself

Call `submitDiagnosticEvent` on the `experimental` member of the server to report something you caught:

```
class OrderEndpoint extends Endpoint {
  Future<String> placeOrder(Session session) async {
    try {
      throw Exception('An exception is thrown');
    } catch (e, stackTrace) {
      session.serverpod.experimental.submitDiagnosticEvent(
        ExceptionEvent(e, stackTrace),
        session: session,
      );
    }
    return 'success';
  }
}
```

The `ExceptionEvent` class is the built-in event for a thrown exception and a stack trace. Passing the session attaches its context to the event, so the handler knows which call the exception came from. This works anywhere you have a session, including endpoint methods, web calls, and future calls.

Related

- [Logging](#): what the server records without any handler.
- [Error handling and exceptions](#): how exceptions reach your app.
- [Run code on shutdown](#): the other experimental API Serverpod exposes.
- [Configuration](#): opting in to experimental features.
- [Testing](#): testing that your handlers receive the events you expect.